Estructuras de Datos no Lineales

1.5. Búsqueda

José Fidel Argudo Argudo
José Antonio Alonso de la Huerta
Mª Teresa García Horcajadas

Versión 3.1

1.5. Búsqueda

1.5.1. Árboles binarios de búsqueda

1.5.2. Árboles binarios de búsqueda equilibrados

1.5.3. Árboles B

1.5.4. Tablas de dispersión

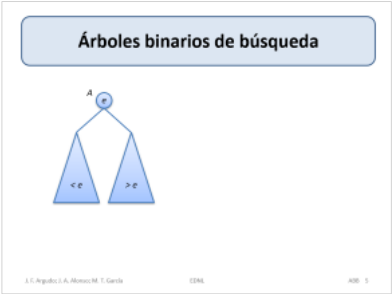
Estructuras de Datos no Lineales

1.5.1. Árboles binarios de búsqueda

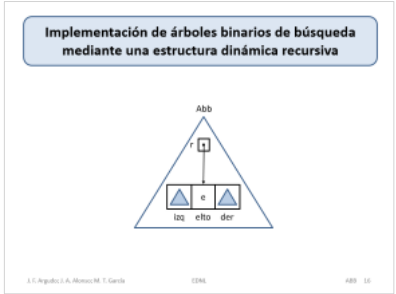
José Fidel Argudo Argudo
José Antonio Alonso de la Huerta
Mª Teresa García Horcajadas

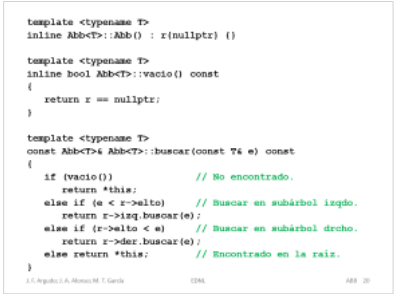
Versión 3.1

Contenido


Árboles binarios de búsqueda
A
e
< e
> e

TAD Árbol binario de búsqueda
Definición:
Un árbol binario de búsqueda es un árbol binario en el que los nodos almacenan elementos de un conjunto (no existen elementos repetidos). La propiedad que define a estos árboles es que todos los elementos almacenados en el subárbol izquierdo de cualquier nodo n son menores que el elemento de n, y todos los elementos almacenados en el subárbol derecho de n son mayores que el elemento almacenado en el mismo.
Consideraremos que existe un orden lineal definido sobre el tipo de los elementos dado por el operador <.

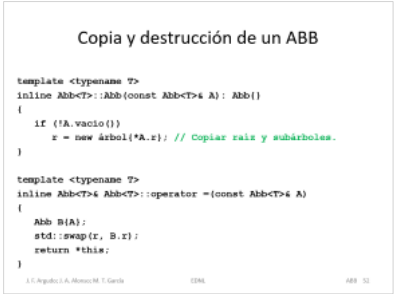

Implementación de árboles binarios de búsqueda mediante una estructura dinámica recursiva
Abb
r
e
izq
elto
der


template <typename T>
inline Abb<T>::Abb() : r(nullptr) {}
template <typename T>
inline bool Abb<T>::vacio() const
{
return r == nullptr;
}
template <typename T>
const Abb<T>& Abb<T>::buscar(const T& e) const
{
if (vacio())
return *this;
else if (e < r->elto)
return r->izq.buscar(e);
else if (r->elto < e)
return r->der.buscar(e);
else return *this;
}

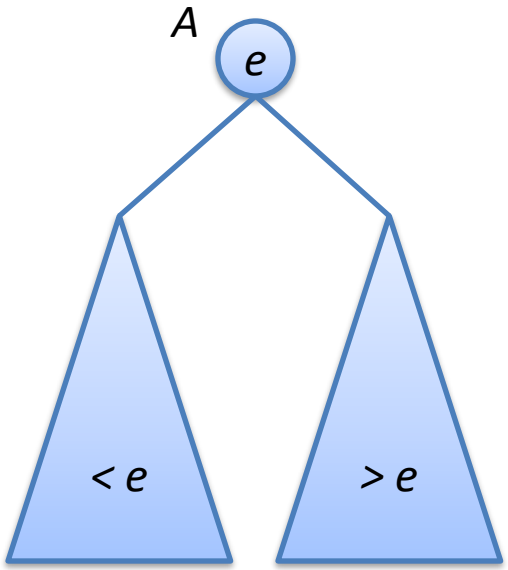
Inserción en un ABB
Insertar 35
30
18
45
9
25
39
57
3
21
28
42
51
6
48
56

Eliminación en un ABB
Caso 1: Suprimir una hoja
Suprimir 28
30
18
45
9
25
39
57
3
21
28
42
51
6
48
56

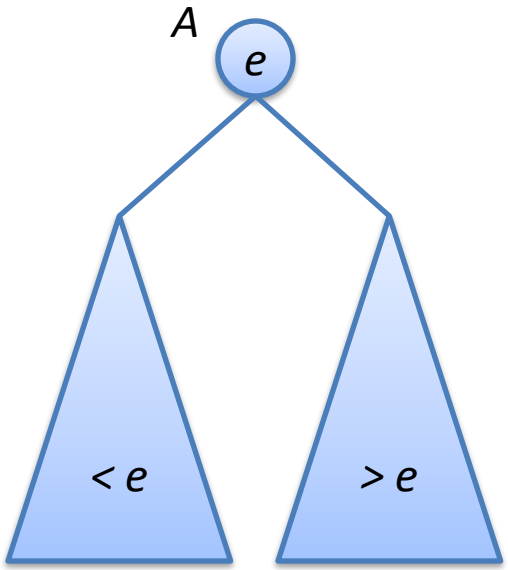
template <typename T>
inline const T& Abb<T>::elemento() const
{
assert(!vacio());
return r->elto;
}
template <typename T>
inline const Abb<T>& Abb<T>::izqdo() const
{
assert(!vacio());
return r->izq;
}
template <typename T>
inline const Abb<T>& Abb<T>::drcho() const
{
assert(!vacio());
return r->der;
}

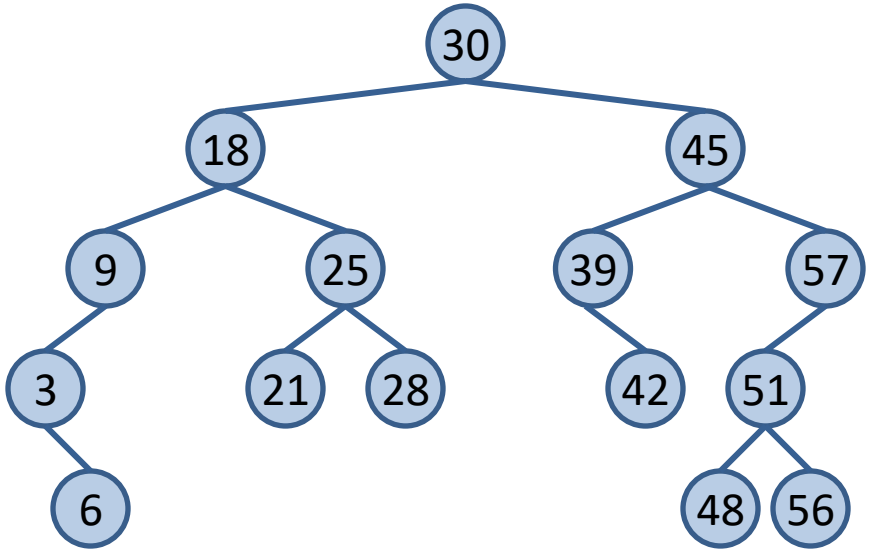

Copia y destrucción de un ABB
template <typename T>
inline Abb<T>::Abb(const Abb<T>& A): Abb()
{
if (!A.vacio())
r = new árbol(*A.r); // Copiar raiz y subárboles.
}
template <typename T>
inline Abb<T>& Abb<T>::operator =(const Abb<T>& A)
{
Abb B(A);
std::swap(r, B.r);
return *this;
}

Árboles binarios de búsqueda

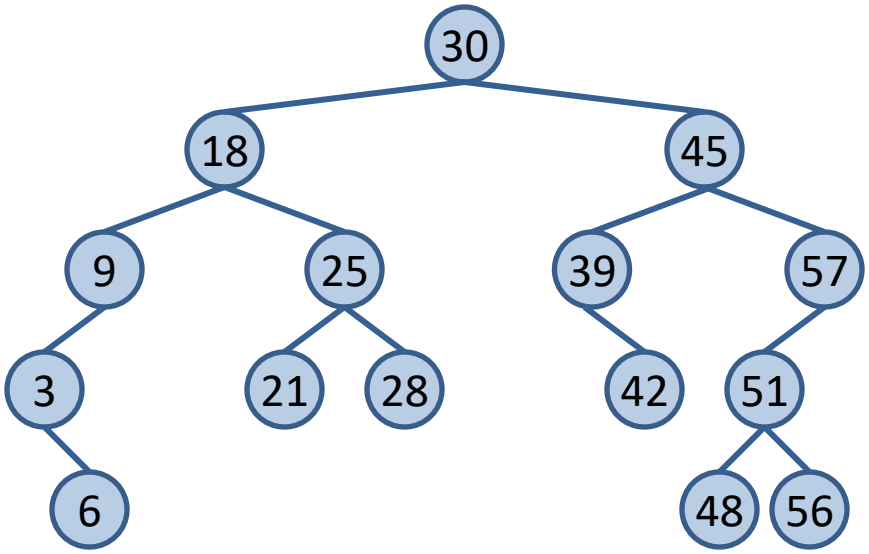


Árboles binarios de búsqueda

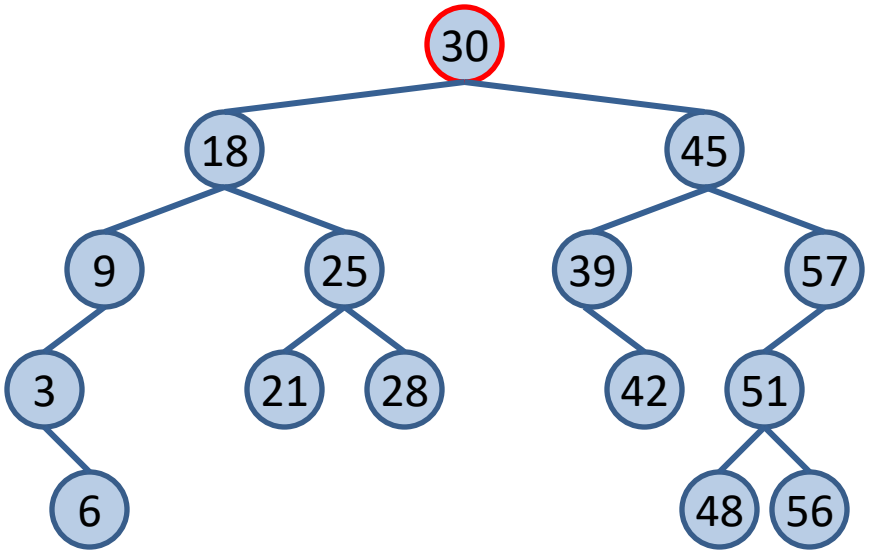




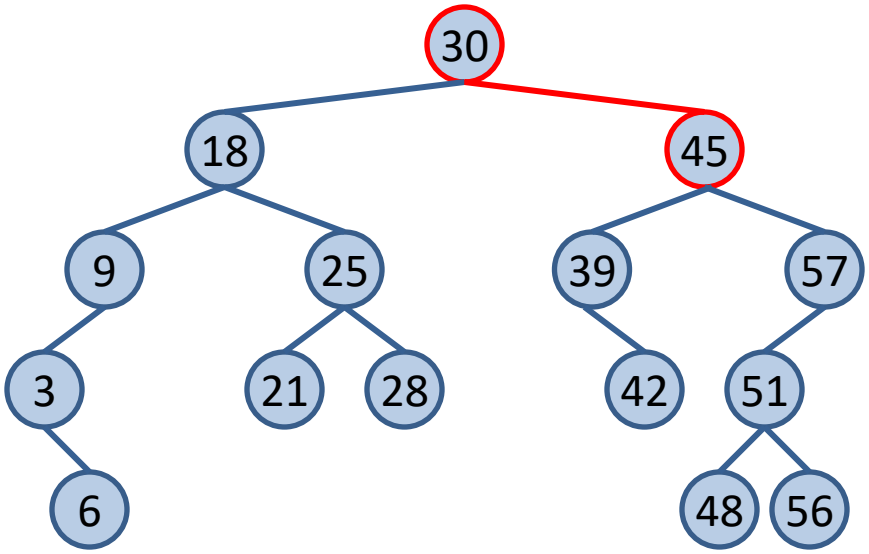
Buscar 51



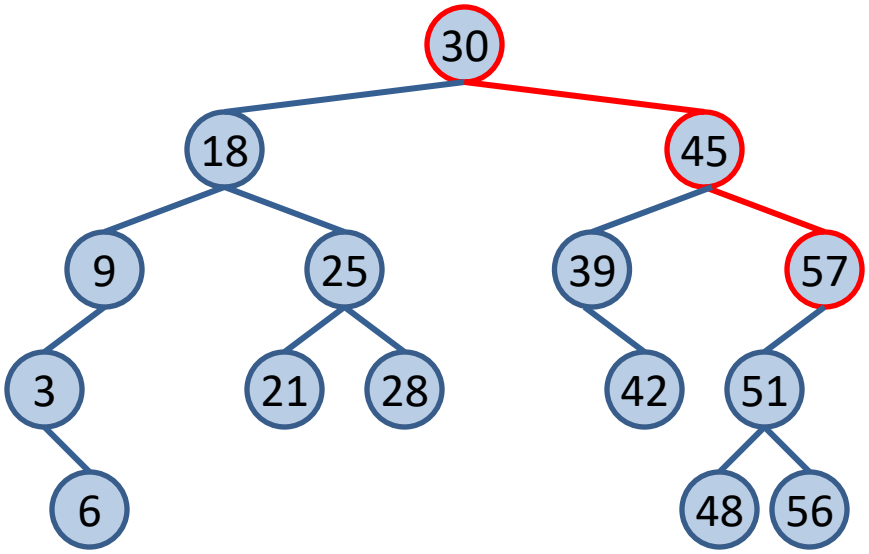
Buscar 51



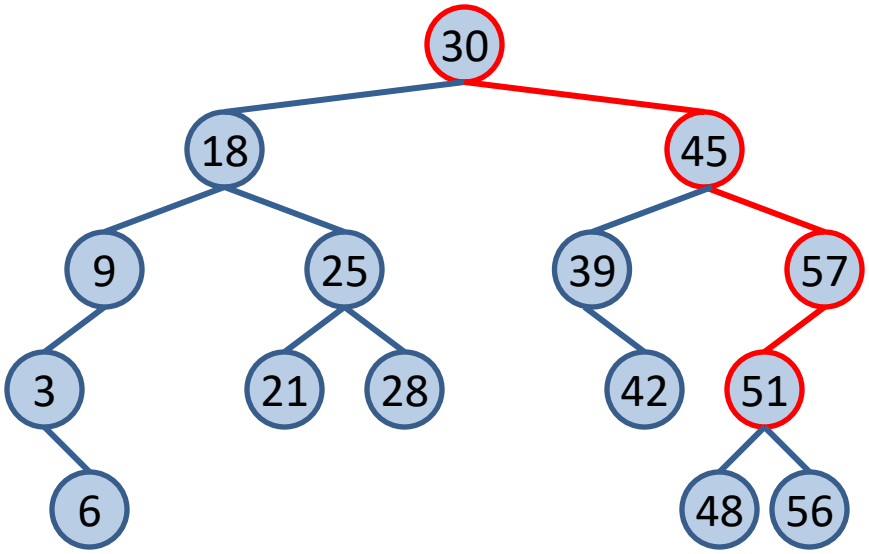
Buscar 51



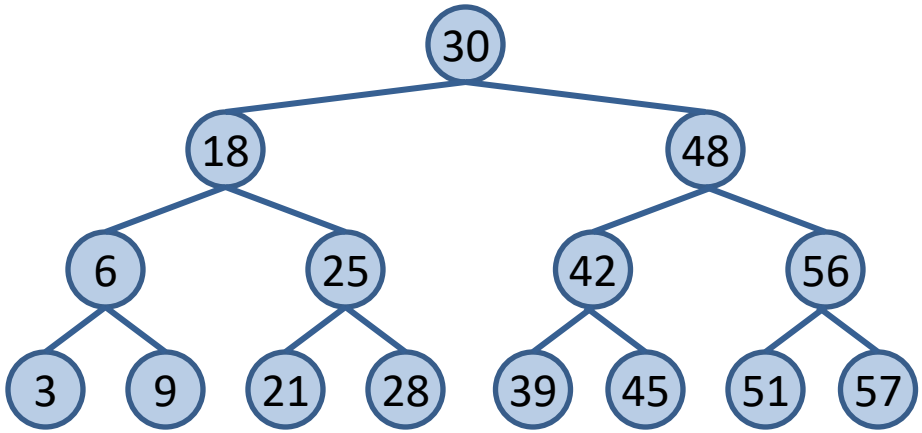
Buscar 51



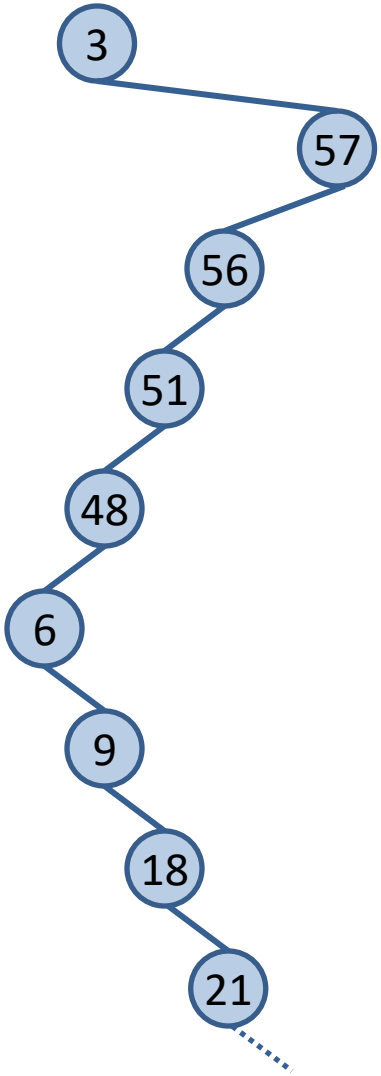
Buscar 51



El tiempo de búsqueda depende de la estructura de ramificación del árbol.



$O(log_2\ n)$



$O(n)$

TAD Árbol binario de búsqueda

Definición:

Un árbol binario de búsqueda es un árbol binario en el que los nodos almacenan elementos de un conjunto (no existen elementos repetidos). La propiedad que define a estos árboles es que todos los elementos almacenados en el subárbol izquierdo de cualquier nodo *n* son menores que el elemento de *n*, y todos los elementos almacenados en el subárbol derecho de *n* son mayores que el elemento almacenado en el mismo.

Consideraremos que existe un orden lineal definido sobre el tipo de los elementos dado por el operador <.

Operaciones:

Abb()

Post: Construye un árbol binario de búsqueda vacío.

const Abb& buscar(const T& e) const

Post: Si el elemento *e* pertenece al árbol, devuelve el subárbol en cuya raíz se encuentra *e*; en caso contrario, devuelve un árbol vacío.

void insertar(const T& e)

Post: Si *e* no pertenece al árbol, lo inserta; en caso contrario, el árbol no se modifica.

void eliminar(const T& e)

Post: Elimina el elemento *e* del árbol. Si *e* no se encuentra, el árbol no se modifica.

bool vacio() const

Post: Devuelve **`true`** si el árbol está vacío y **`false`** en caso contrario.

const T& elemento() const

Pre: Árbol no vacío.

Post: Devuelve el elemento de la raíz de un árbol binario de búsqueda.

const Abb& izqdo() const

Pre: Árbol no vacío.

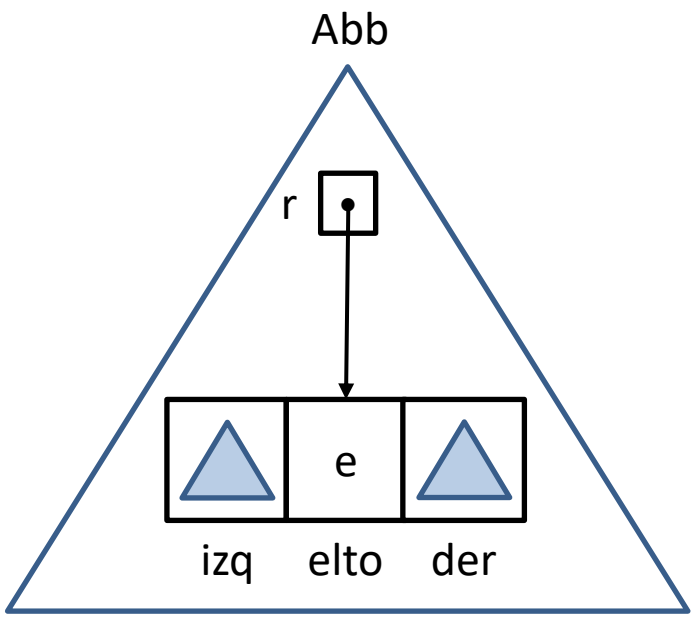
Post: Devuelve el subárbol izquierdo.

const Abb& drcho() const

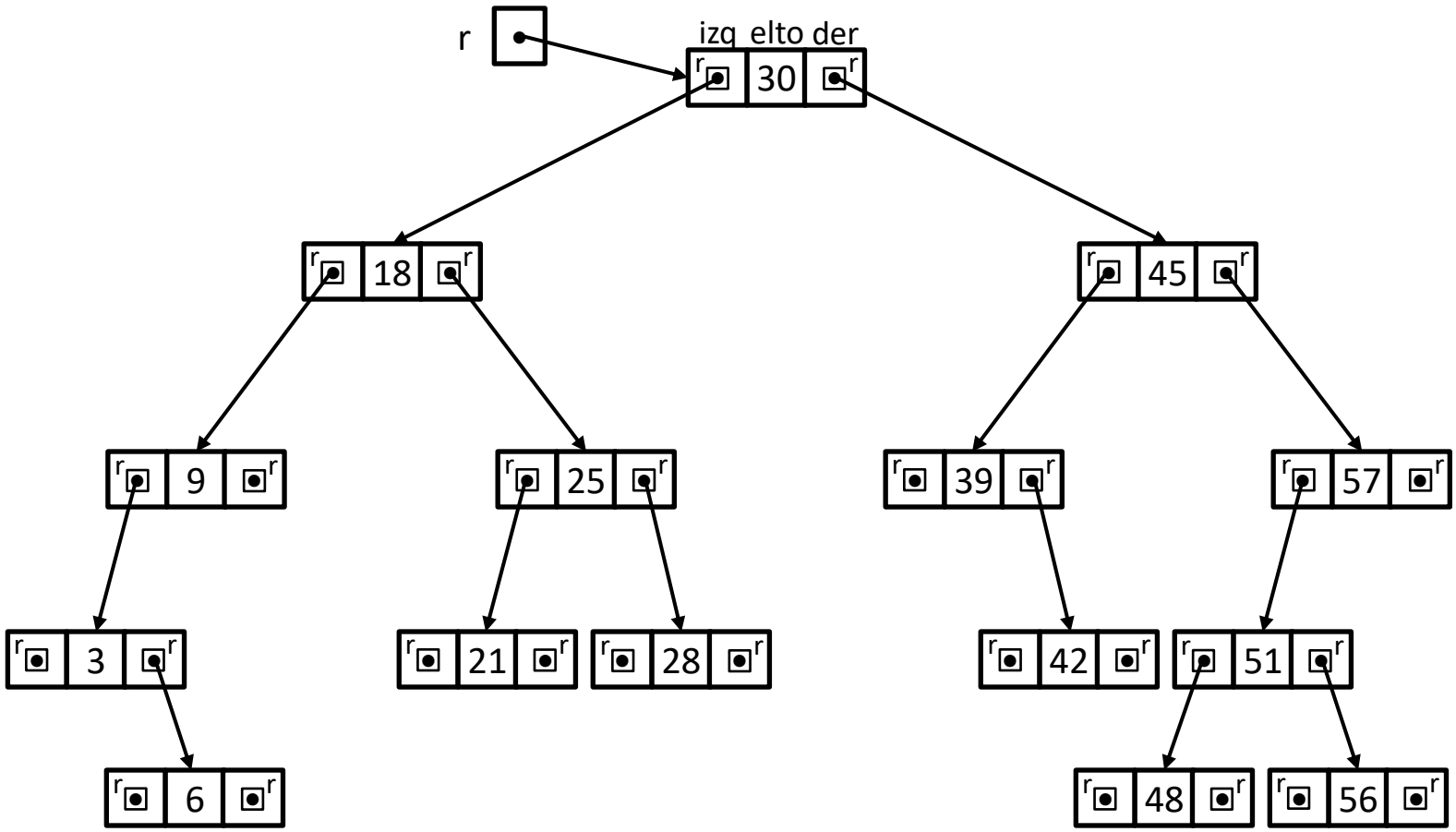
Pre: Árbol no vacío.

Post: Devuelve el subárbol derecho.

Implementación de árboles binarios de búsqueda mediante una estructura dinámica recursiva



Implementación de árboles binarios de búsqueda mediante una estructura dinámica recursiva



```
#ifndef ABB_H
#define ABB_H
#include <cassert>
#include <utility>   // swap

template <typename T> class Abb {
public:
   Abb();
   const Abb& buscar(const T& e) const;
   void insertar(const T& e);
   void eliminar(const T& e);
   bool vacio() const;
   const T& elemento() const;
   const Abb& izqdo() const;
   const Abb& drcho() const;
   Abb(const Abb& A);                    // Ctor. de copia
   Abb& operator =(const Abb& A);        // Asig. árboles
   ~Abb();                               // Destructor
```

```
private:
   struct arbol {
      T elto;
      Abb izq, der;

      arbol(const arbol& a) = default;
      ~arbol() = default;
   };

   arbol* r;   // Raíz del árbol
};
```

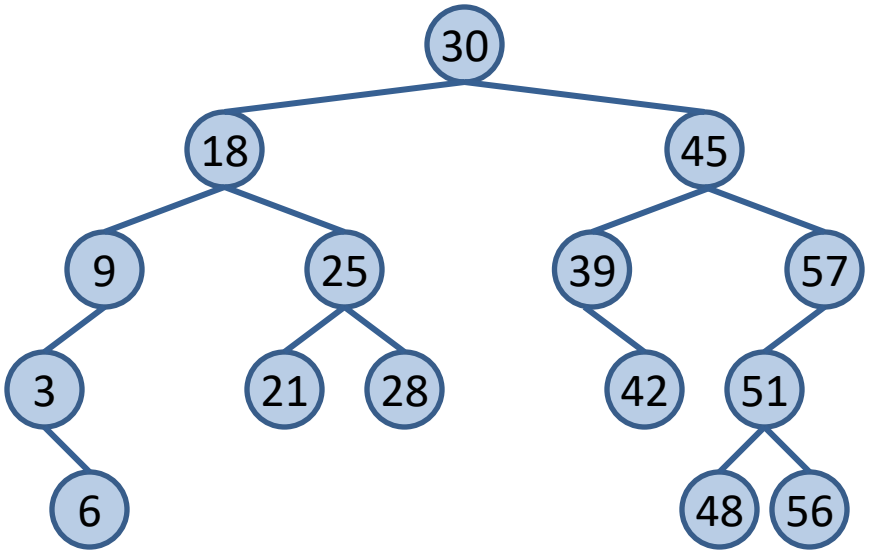
```
template <typename T>
inline Abb<T>::Abb() : r{nullptr} {}

template <typename T>
inline bool Abb<T>::vacio() const
{
   return r == nullptr;
}

template <typename T>
const Abb<T>& Abb<T>::buscar(const T& e) const
{
   if (vacio())                  // No encontrado.
      return *this;
   else if (e < r->elto)         // Buscar en subárbol izqdo.
      return r->izq.buscar(e);
   else if (r->elto < e)         // Buscar en subárbol drcho.
      return r->der.buscar(e);
   else return *this;            // Encontrado en la raíz.
}
```

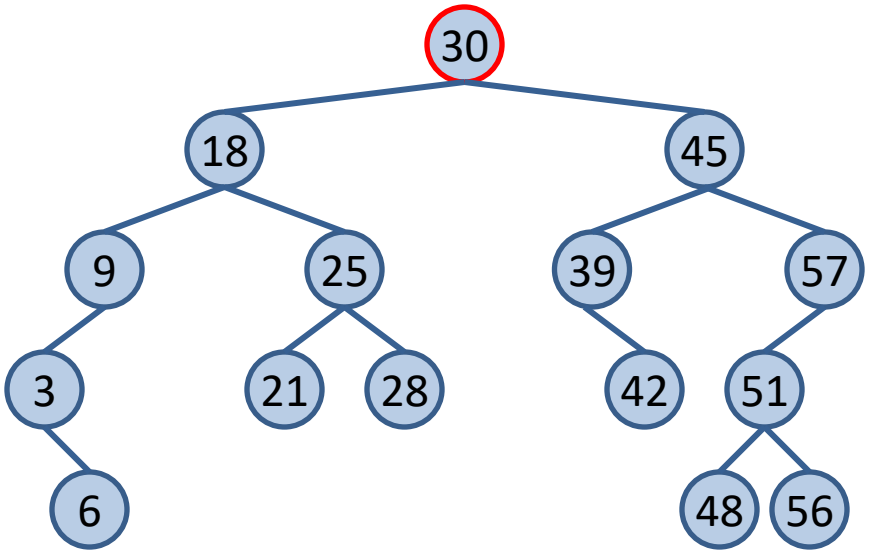
Inserción en un ABB

Insertar 35



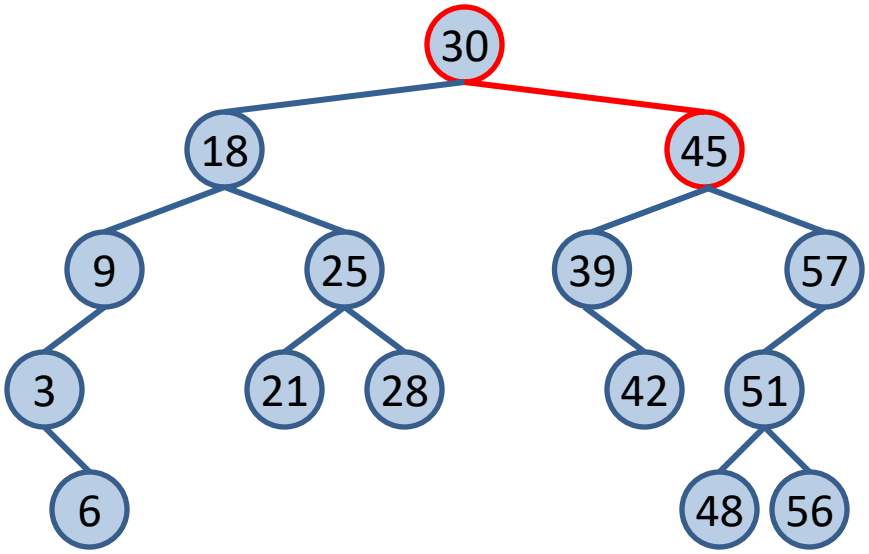
Inserción en un ABB

Insertar 35



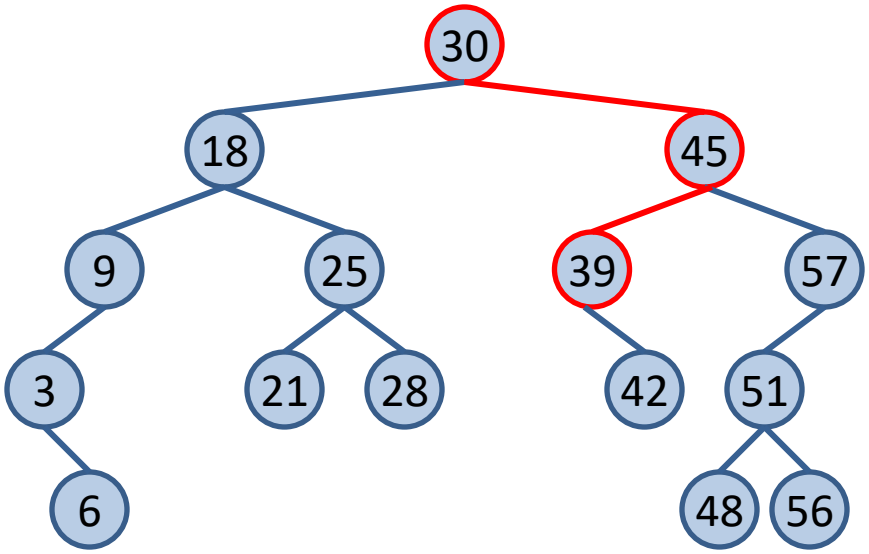
Inserción en un ABB

Insertar 35



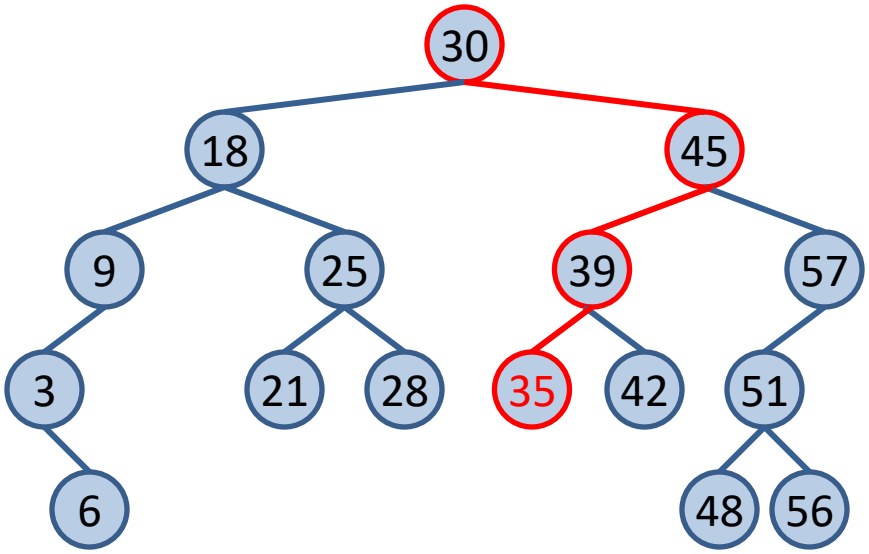
Inserción en un ABB

Insertar 35



Inserción en un ABB

Insertar 35

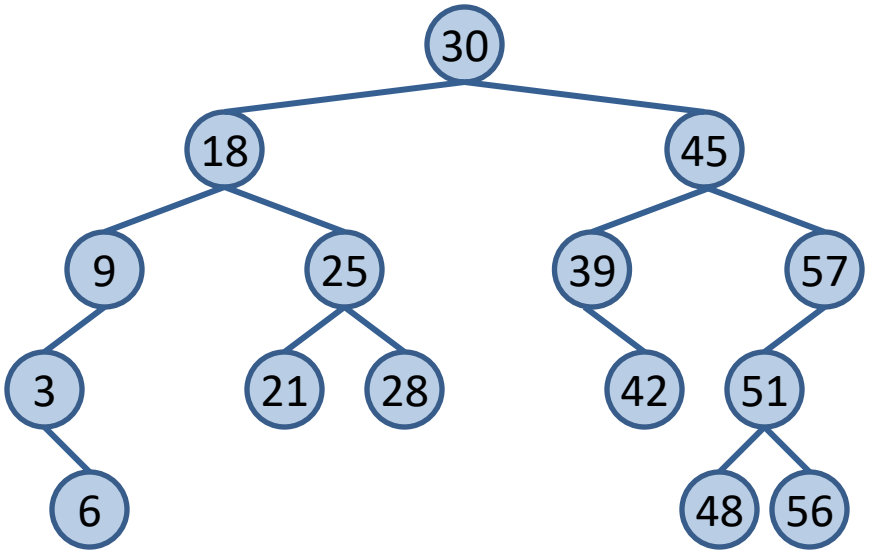


```
template <typename T>
void Abb<T>::insertar(const T& e)
{
   if (vacio())                // Insertar en la raíz.
      r = new árbol{e};
   else if (e < r->elto)       // Insertar en subárbol izqdo.
      r->izq.insertar(e);
   else if (r->elto < e)       // Insertar en subárbol drcho.
      r->der.insertar(e);
}
```

Eliminación en un ABB

Caso 1: Suprimir una hoja

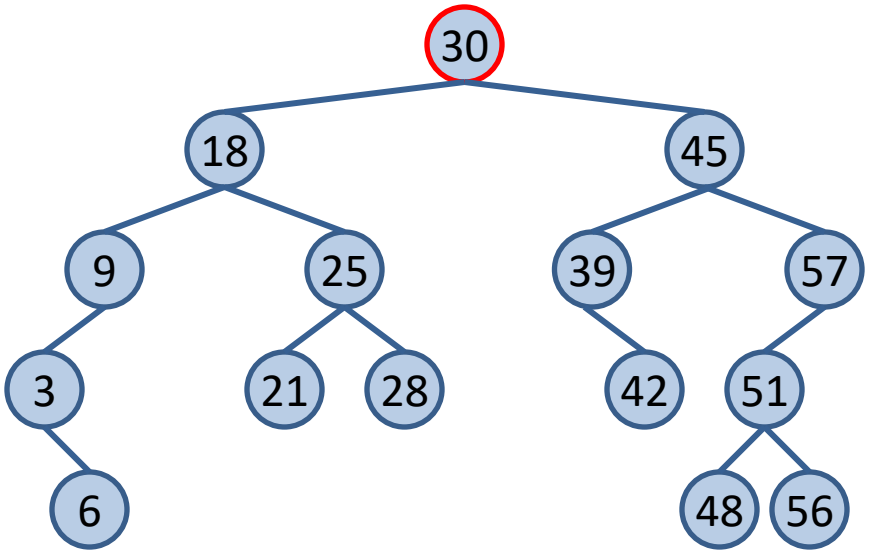
Suprimir 28



Eliminación en un ABB

Caso 1: Suprimir una hoja

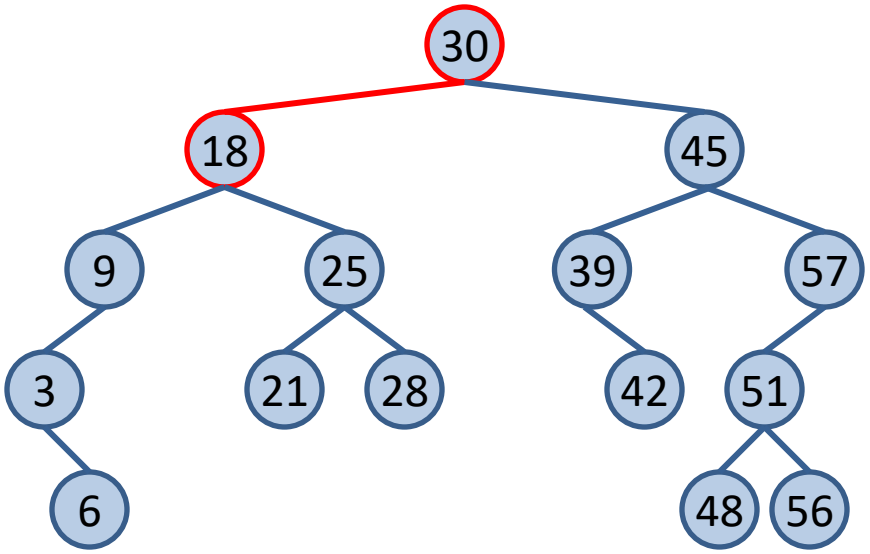
Suprimir 28



Eliminación en un ABB

Caso 1: Suprimir una hoja

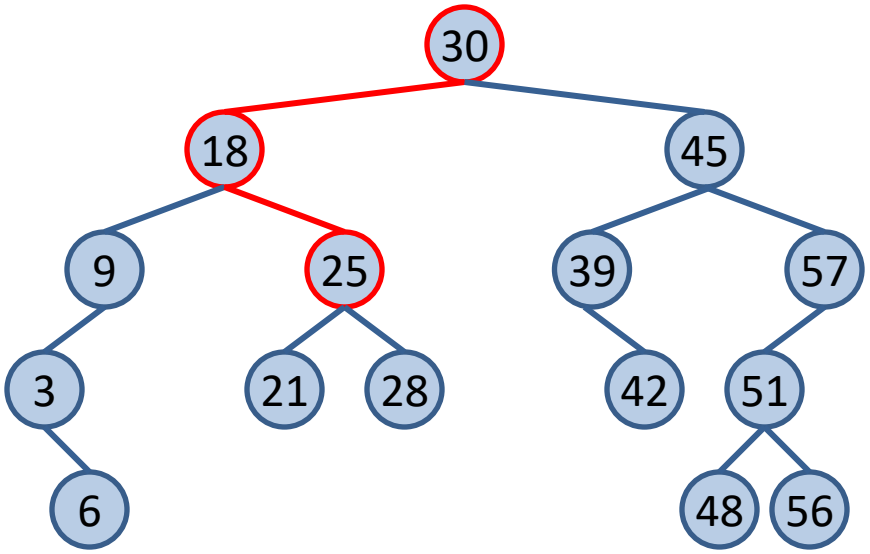
Suprimir 28



Eliminación en un ABB

Caso 1: Suprimir una hoja

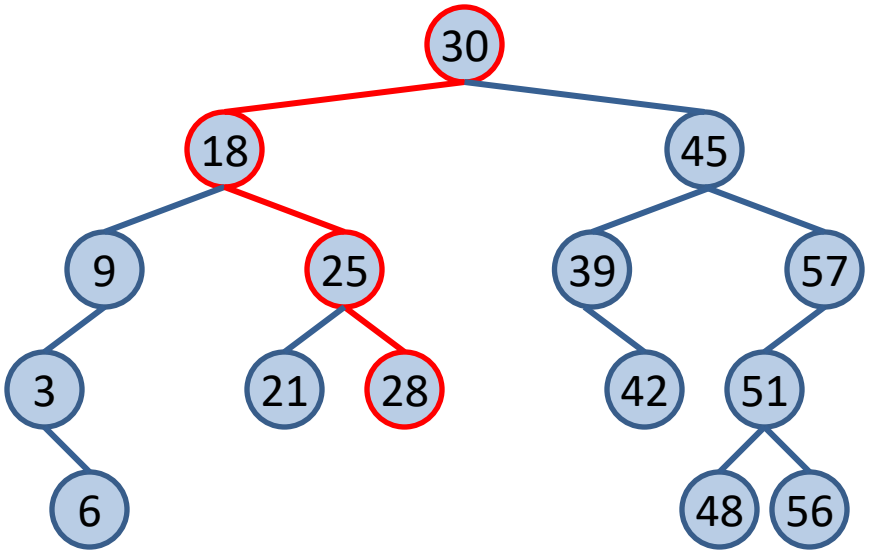
Suprimir 28



Eliminación en un ABB

Caso 1: Suprimir una hoja

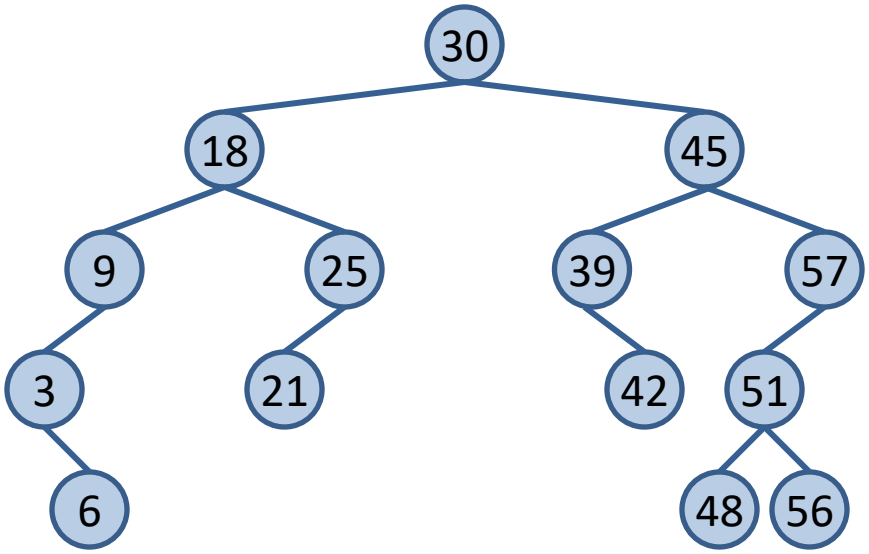
Suprimir 28



Eliminación en un ABB

Caso 1: Suprimir una hoja

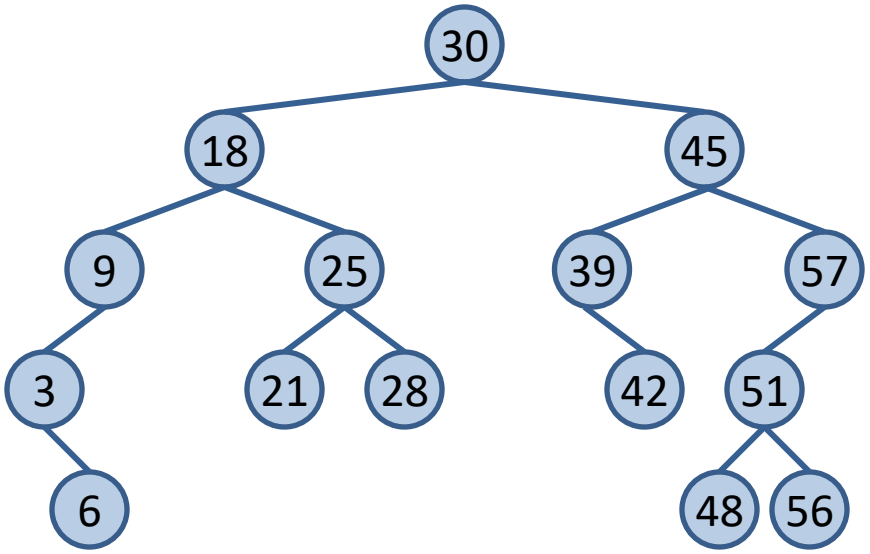
Suprimir 28



Eliminación en un ABB

Caso 2: Suprimir un nodo con sólo hijo izquierdo

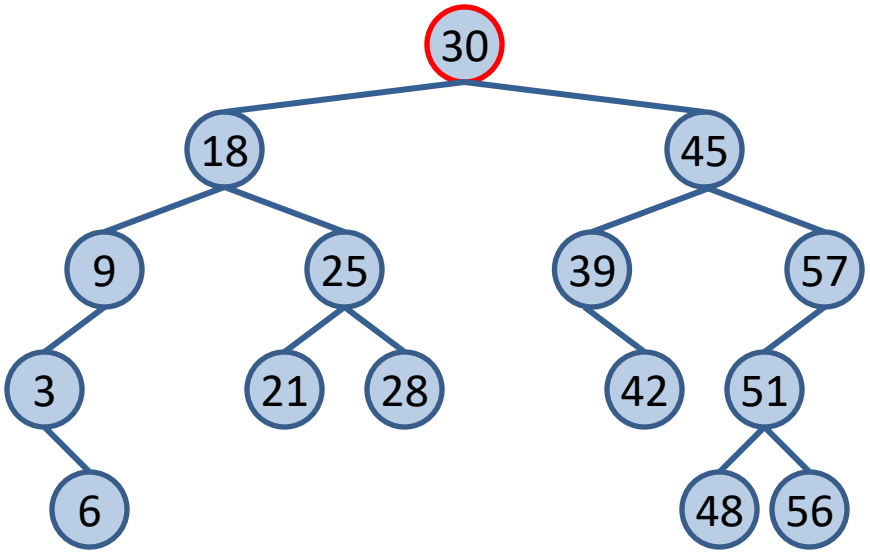
Suprimir 9



Eliminación en un ABB

Caso 2: Suprimir un nodo con sólo hijo izquierdo

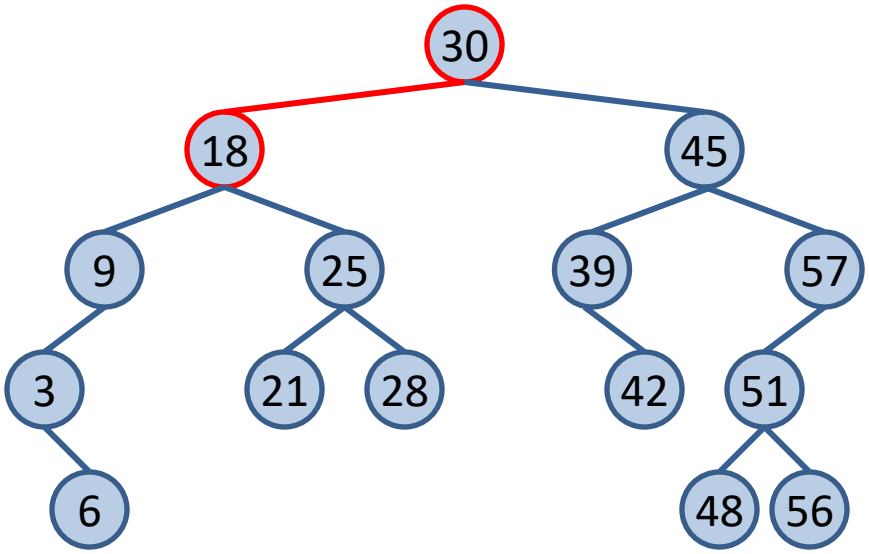
Suprimir 9



Eliminación en un ABB

Caso 2: Suprimir un nodo con sólo hijo izquierdo

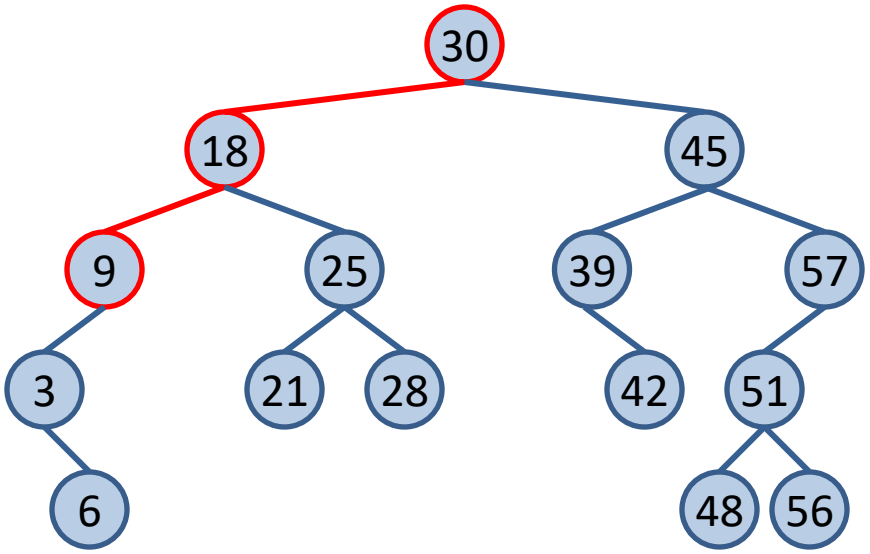
Suprimir 9



Eliminación en un ABB

Caso 2: Suprimir un nodo con sólo hijo izquierdo

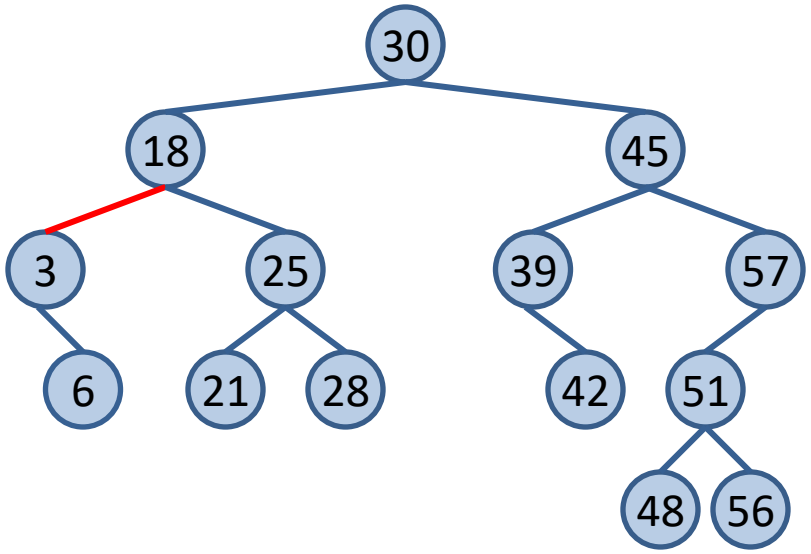
Suprimir 9



Eliminación en un ABB

Caso 2: Suprimir un nodo con sólo hijo izquierdo

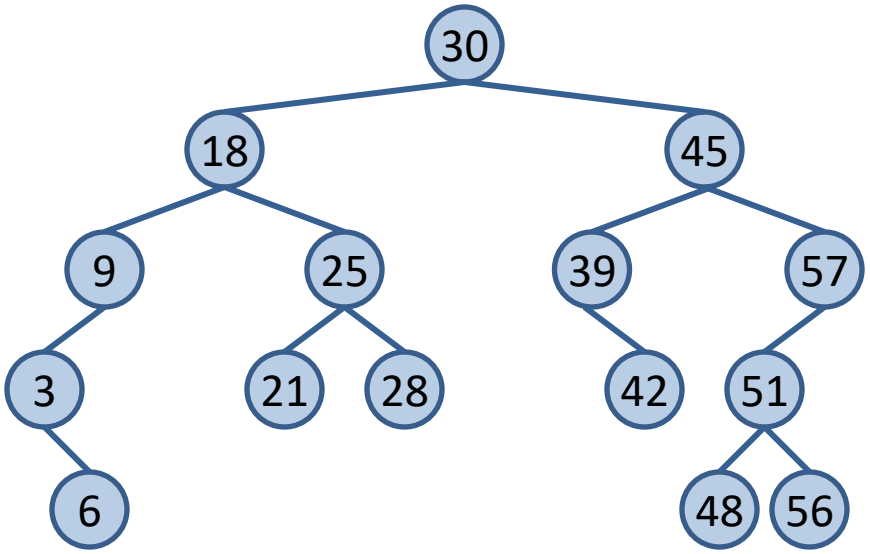
Suprimir 9



Eliminación en un ABB

Caso 3: Suprimir un nodo con sólo hijo derecho

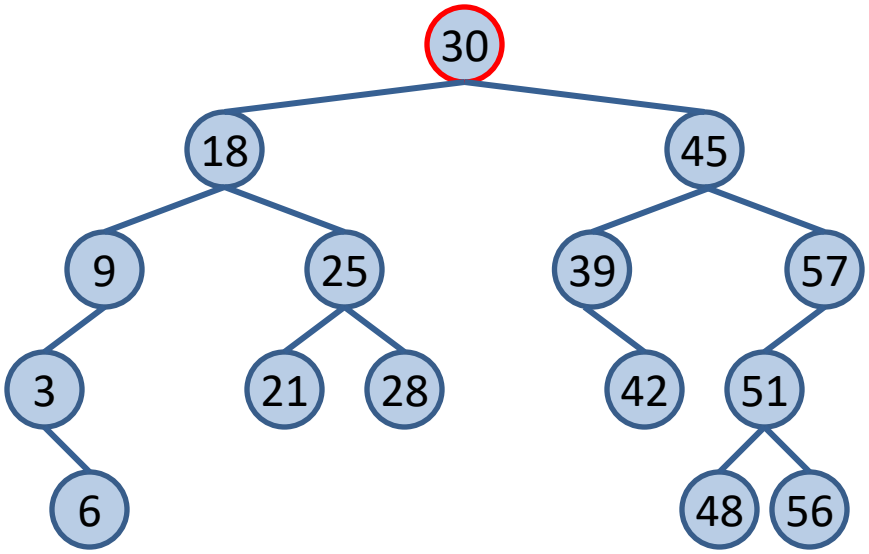
Suprimir 39



Eliminación en un ABB

Caso 3: Suprimir un nodo con sólo hijo derecho

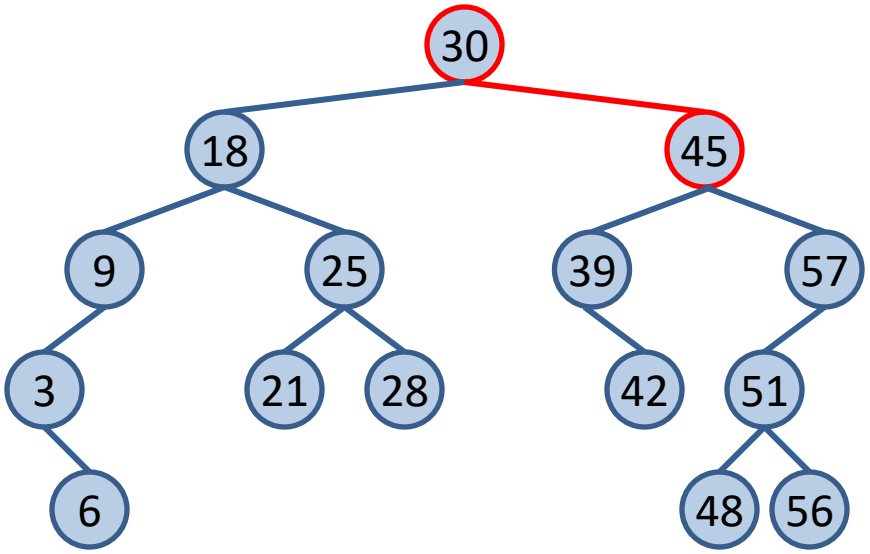
Suprimir 39



Eliminación en un ABB

Caso 3: Suprimir un nodo con sólo hijo derecho

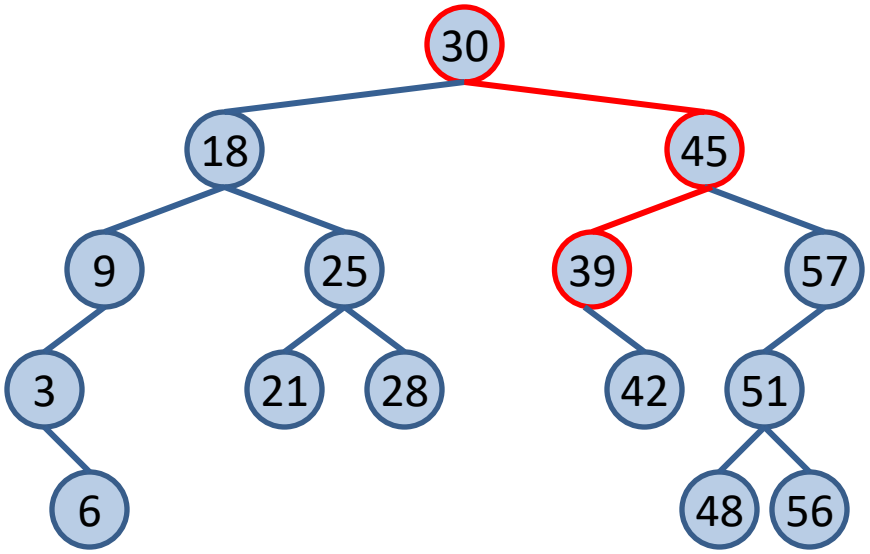
Suprimir 39



Eliminación en un ABB

Caso 3: Suprimir un nodo con sólo hijo derecho

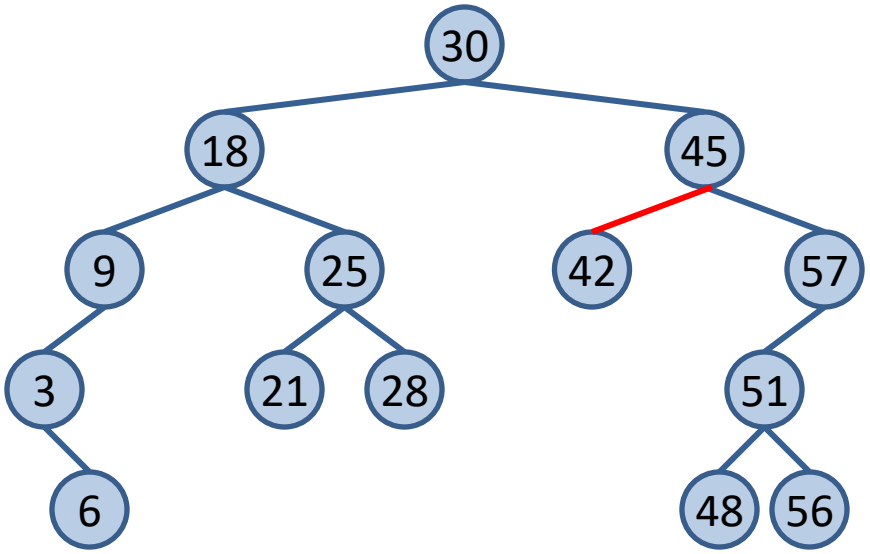
Suprimir 39



Eliminación en un ABB

Caso 3: Suprimir un nodo con sólo hijo derecho

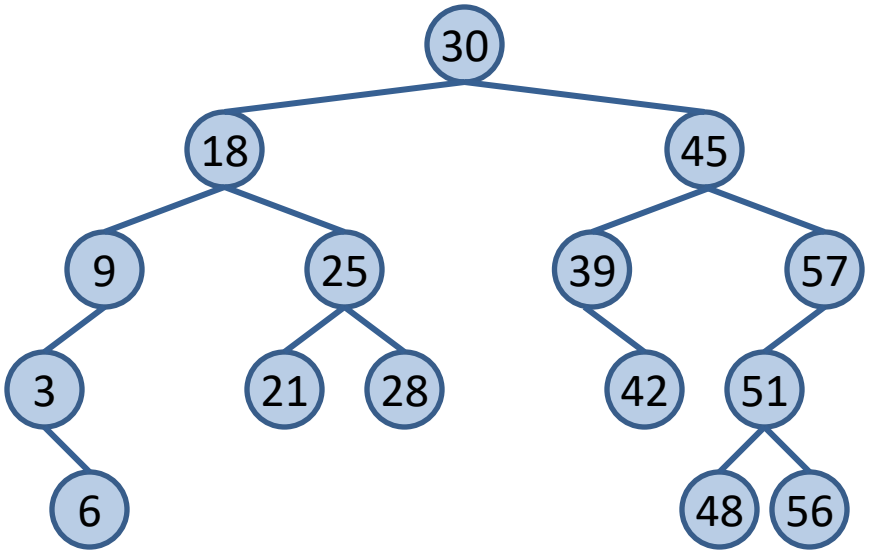
Suprimir 39



Eliminación en un ABB

Caso 4: Suprimir un nodo con dos hijos

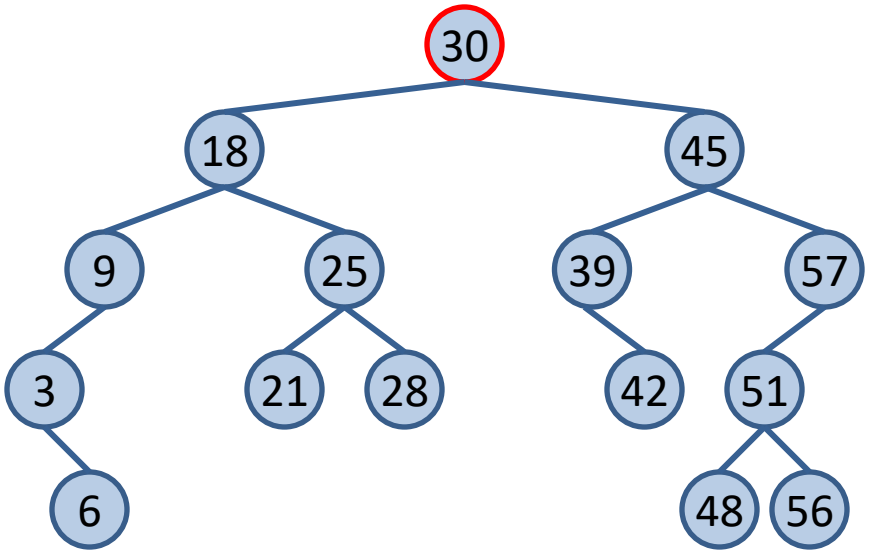
Suprimir 30



Eliminación en un ABB

Caso 4: Suprimir un nodo con dos hijos

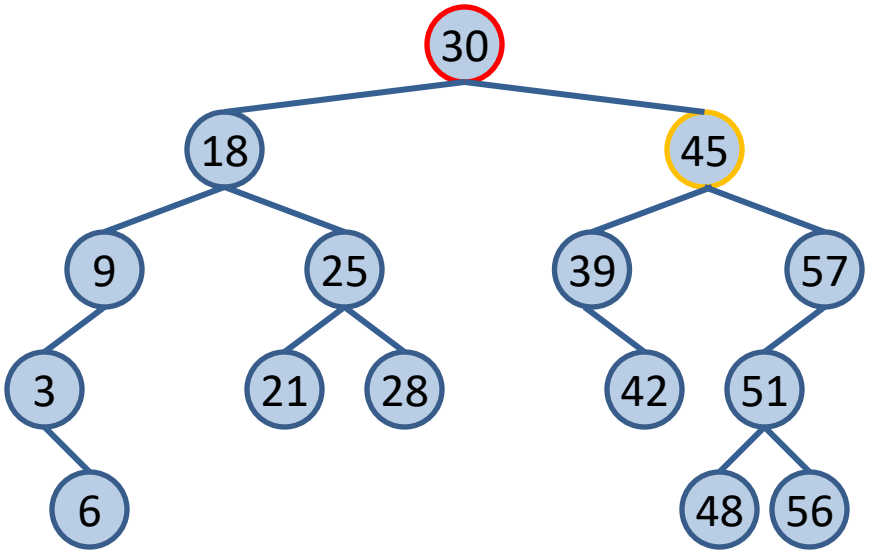
Suprimir 30



Eliminación en un ABB

Caso 4: Suprimir un nodo con dos hijos

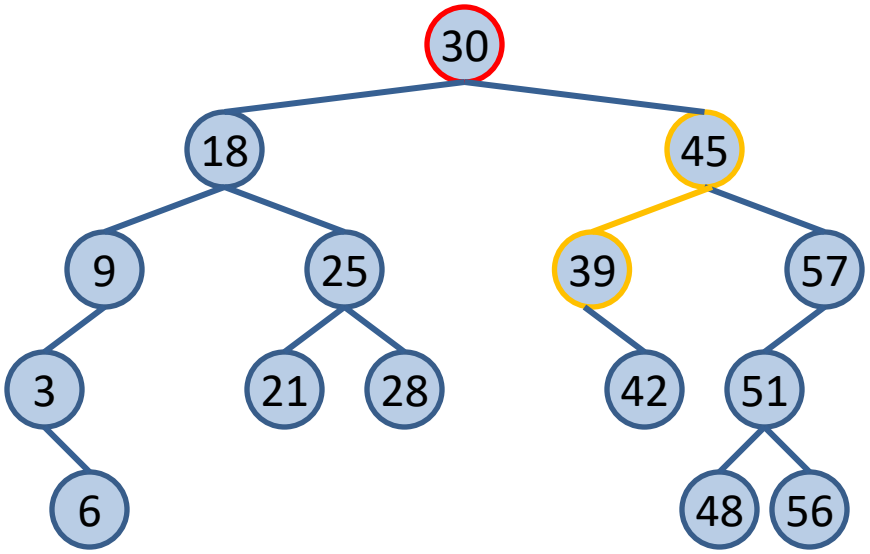
Suprimir 30



Eliminación en un ABB

Caso 4: Suprimir un nodo con dos hijos

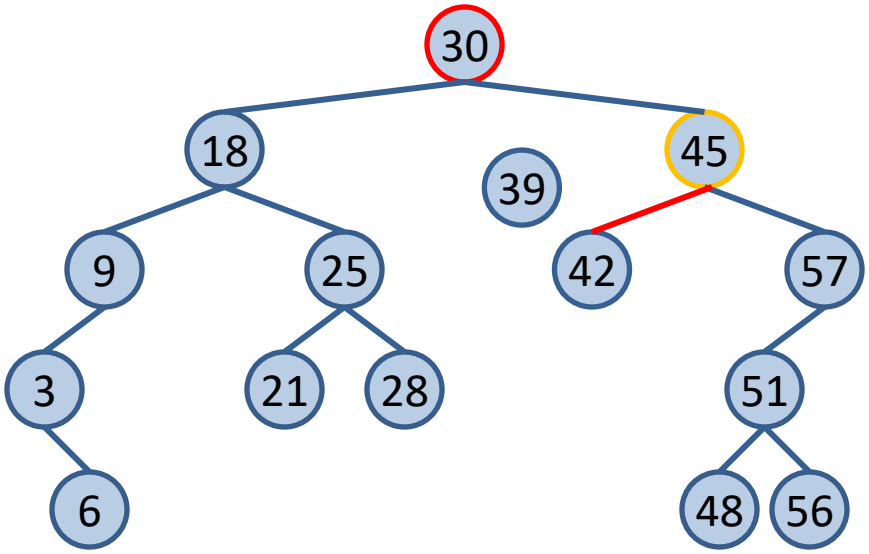
Suprimir 30



Eliminación en un ABB

Caso 4: Suprimir un nodo con dos hijos

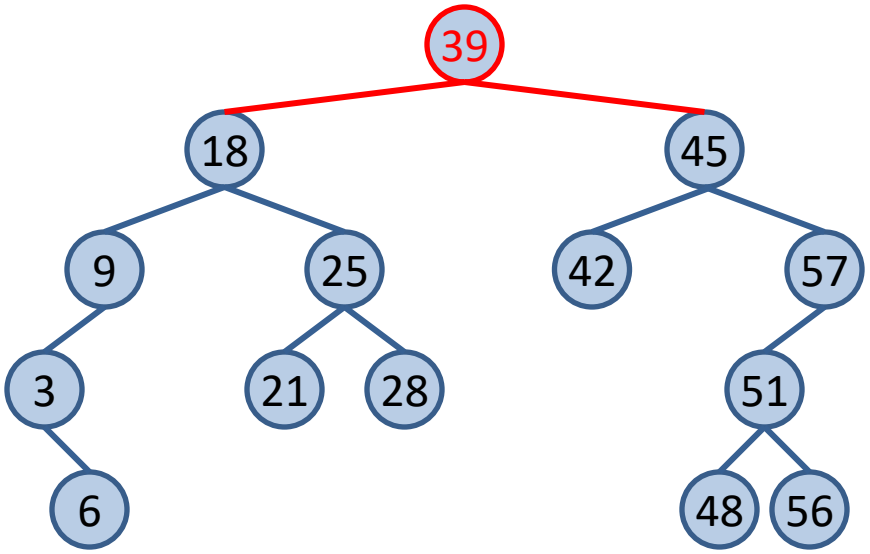
Suprimir 30



Eliminación en un ABB

Caso 4: Suprimir un nodo con dos hijos

Suprimir 30



```
template <typename T>
void Abb<T>::eliminar(const T& e)
{
   if (!vacio()) {
      if (e < r->elto)          // Eliminar del subárbol izqdo.
         r->izq.eliminar(e);
      else if (r->elto < e)     // Eliminar del subárbol drcho.
         r->der.eliminar(e);
      else {                    // Eliminar la raíz.
         árbol* a;   // Sustituto de la raíz.
         if (!r->izq.r && !r->der.r) // 1. La raíz es hoja.
           a = nullptr;
         else if (!r->der.r) { // 2. Raíz sólo tiene hijo izqdo.
           a = r->izq.r;
           r->izq.r = nullptr; // Descolgar el subárbol izqdo.
         }
```

```
        else if (!r->izq.r) { // 3. Raíz sólo tiene hijo drcho.
          a = r->der.r;
          r->der.r = nullptr; // Descolgar el subárbol drcho.
        }
        else {                // 4. Raíz tiene dos hijos.
          // Buscar el mínimo del subárbol drcho. de la raíz.
          Abb* pMin = &r->der;
          while (pMin->r->izq.r != nullptr) pMin = &pMin->r->izq;
          // Extraer el mínimo y sustituirlo por su hijo drcho.
          a = pMin->r;
          pMin->r = a->der.r;
          // Transferir al mínimo los subárboles de la raíz.
          a->izq.r = r->izq.r; // Colgar de *a los
          a->der.r = r->der.r; // subárboles de la raíz.
          r->izq.r = nullptr;  // Descolgarlos
          r->der.r = nullptr;  // de la raíz.
        }
        delete r; // Destruir la raíz.
        r = a;    // Sustituir la raíz eliminada.
      } // Eliminar la raíz.
    } // if (!vacio())
}
```

```
template <typename T>
inline const T& Abb<T>::elemento() const
{
   assert(!vacio());
   return r->elto;
}

template <typename T>
inline const Abb<T>& Abb<T>::izqdo() const
{
   assert(!vacio());
   return r->izq;
}

template <typename T>
inline const Abb<T>& Abb<T>::drcho() const
{
   assert(!vacio());
   return r->der;
}
```

Copia y destrucción de un ABB

```
template <typename T>
inline Abb<T>::Abb(const Abb<T>& A): Abb{}
{
   if (!A.vacio())
      r = new árbol{*A.r}; // Copiar raíz y subárboles.
}

template <typename T>
inline Abb<T>& Abb<T>::operator =(const Abb<T>& A)
{
   Abb B{A};
   std::swap(r, B.r);
   return *this;
}
```

```
template <typename T>
Abb<T>::~Abb()
{
   delete r;     // Destruir raíz y subárboles con r->~arbol()
   r = nullptr; // El árbol queda vacío.
}

#endif // ABB_H
```